SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I _Lavanya.M (192321051)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Hospital Doctor-Shift Assignment using Backtracking

Problem

There are 3 doctors:

- D1
- D2
- D3

and 3 shifts:

- Morning (M)
- Afternoon (A)
- Night (N)

Constraints

1. D1 cannot work Night.
2. D2 must work before D3.
3. Only one doctor per shift.
4. D3 cannot work Morning.
5. Every doctor must receive exactly one shift.

Step 1: Define Variables

The variables are:

$$D1, D2, D3$$

Domain

Each doctor can initially be assigned:

$$\{M, A, N\}$$

Step 2: Apply Constraints

Constraint 1

D1 cannot work Night.

$$D1 \neq N$$

Therefore:

$$D1 \in \{M, A\}$$

Constraint 2

D2 must work before D3.

Therefore:

$$Shift(D2) < Shift(D3)$$

where:

$$M < A < N$$

Constraint 3

Only one doctor per shift.

Therefore:

$$D1 \neq D2 \neq D3$$

Constraint 4

D3 cannot work Morning.

$$D3 \neq M$$

Step-by-Step Backtracking

We begin with:

D1 = ?

D2 = ?

D3 = ?

Step 1: Assign D1 = Morning

Check:

- D1 is not Night → ✓
- Morning is available → ✓

So:
D1 = Morning
Remaining shifts:
Afternoon, Night

Step 2: Assign D2 = Afternoon
Check:

- Afternoon is available → ✓
- D2 must work before D3 → D3 must be Night ✓

So:
D1 = Morning
D2 = Afternoon
D3 = ?

Step 3: Assign D3 = Night
Check:

- D3 cannot work Morning → Night is allowed ✓
- D2 must work before D3:

$$Afternoon < Night$$

✓ Satisfied.

- Only one doctor per shift → ✓
- Every doctor has exactly one shift → ✓

Therefore:
D1 = Morning
D2 = Afternoon
D3 = Night

Constraint Checking Table

Doctor	Assigned Shift	Constraint Check	Result
D1	Morning	D1 ≠ Night	✓
D2	Afternoon	D2 before D3	✓
D3	Night	D3 ≠ Morning	✓
All	Different shifts	One doctor per shift	✓
All	One shift each	Complete assignment	✓

Final Valid Schedule

Shift	Doctor
Morning	**D1**
Afternoon	**D2**
Night	**D3**

Result
Using **Backtracking Search**, a valid assignment is:

$$\boxed{D1 \rightarrow Morning,\ D2 \rightarrow Afternoon,\ D3 \rightarrow Night}$$

All constraints are satisfied.

2. Robot Grid Search using BFS

Problem

The robot operates in a **5 × 5 grid** and must move from **S (Start)** to **G (Goal)**.

Constraints

1. Movement: Up, Down, Left and Right.
2. Each movement has cost 1.
3. Obstacles cannot be crossed.
4. Manhattan distance is given as the heuristic.

Important Point

There is a small terminology issue in the question:

BFS does not use a heuristic or a priority queue.

- **BFS** uses a **FIFO queue**.
- **Greedy Best-First Search** uses the Manhattan heuristic.
- **A*** uses both actual path cost and Manhattan heuristic.

For a strict **BFS answer**, we should use:

$$f(n) = \text{depth/path cost}$$

and expand nodes level by level.

If the examiner specifically expects the **Manhattan heuristic**, then A* would normally be the appropriate algorithm:

$$f(n) = g(n) + h(n)$$

where:

$$g(n) = \text{cost from S to n}$$

and

$$h(n) = | x_n - x_G | + | y_n - y_G |$$

BFS Algorithm

START

Put S into the queue
Mark S as visited

WHILE queue is not empty:

  Remove the first node from queue

  IF node = G:
    Return path and cost

  Generate Up, Down, Left and Right neighbours

  For each valid neighbour:
    If not visited and not an obstacle:
      Mark as visited
      Store parent
      Add to queue

END

BFS Expansion Pattern
BFS explores the grid level by level:
Level 0:
S

Level 1:
All valid cells one move from S

Level 2:
All valid cells two moves from S

Level 3:
All valid cells three moves from S

...
Because every move has cost **1**, BFS finds the path containing the **minimum number of moves**.
Cost
If the solution requires k moves:

$$\boxed{Cost = k}$$

Manhattan Distance
For a cell:

$$(x, y)$$

and goal:

$$(x_G, y_G)$$

the Manhattan distance is:

$$\boxed{h(n) = | x - x_G | + | y - y_G |}$$

Example
If the robot is at (2,3) and G is at (5,5):

$$h(n) = | 2 - 5 | + | 3 - 5 |$$
$$= 3 + 2$$
$$= \boxed{5}$$

Result
BFS explores the grid **level by level** and, because every movement has equal cost, produces an **optimal minimum-move path**.

3. Autonomous Rescue Robot
Given Problem
A rescue robot must travel from **S** to **G** in a disaster-affected building.
The environment contains:

- Obstacles
- Risky zones
- Limited sensing
- Unknown information
- Different movement costs

The robot must find the survivor while minimizing total cost and maintaining safety.

3(a). Constraint Analysis

1. Movement Constraint

The robot can move only:

- Up
- Down
- Left
- Right

It cannot move diagonally.

2. Obstacle Constraint

Blocked cells cannot be entered.

Therefore:

$$Obstacle \Rightarrow \text{Movement not allowed}$$

The robot must search for an alternative route.

3. Limited Sensing

The robot can observe only adjacent cells.

Therefore, the robot **does not know the complete environment initially**.

It must discover the environment while moving.

This makes the problem **partially observable**.

4. Movement Cost

Every normal movement has cost:

$$1$$

5. Risky Zone Cost

Entering a risky zone has an additional cost of:

$$2$$

Therefore:

$$Cost(risky\ movement) = 1 + 2 = 3$$

So the robot should preferably avoid risky areas when a safer path has a lower total cost.

6. Dynamic Knowledge

The robot must continuously update its knowledge.

For example:

Initial knowledge

↓

Observe surroundings

↓

Discover obstacle

↓

Update map

↓

Recalculate route

↓

Move

↓

Observe again

This is an **online search problem**.

3(b). Suitable Search Approach
Uniform Cost Search + Online Search
A suitable solution is a combination of:
Uniform Cost Search (UCS)
and
Online Search Agent

Why UCS?
UCS selects the path with the **lowest cumulative cost**.
For example:
Path 1
S → A → B → G
Cost:

$$1 + 1 + 1 = 3$$

Path 2
S → C → D → G
Suppose C is risky:

$$1 + 3 + 1 = 5$$

UCS chooses:

$$\boxed{Path\ 1}$$

because its total cost is lower.

UCS Formula
The priority of a node is:

$$\boxed{g(n)}$$

where:

$g(n)$ = total cost from S to n

The node with the smallest g(n) is expanded first.

Online Search Process
Because the robot cannot see the complete map, it works continuously.
Step 1
Start at S.
Step 2
Observe adjacent cells.
Step 3
Identify:

- Free cells
- Obstacles
- Risky cells

Step 4
Update the internal map.
Step 5
Calculate the lowest-cost available route.
Step 6
Move to the next cell.
Step 7
Observe the new surroundings.

Step 8
Update the map again.
Step 9
Recalculate if a new obstacle or risky area is discovered.
Step 10
Continue until G is reached.

Simple Pseudocode

```
START

Current = S
KnownMap = empty

WHILE Current != G:

    Observe adjacent cells

    Update KnownMap

    Find reachable cells

    Calculate movement costs

    Use UCS to select lowest-cost route

    Move to next cell

    Update Current

RETURN success
```

3(c). AI Concepts
1. Intelligent Agent
The rescue robot is an **intelligent agent** because it:

- Perceives its environment
- Processes information
- Makes decisions
- Performs actions
- Updates its knowledge

Agent cycle
Perceive → Think → Act → Perceive → Think → Act

2. Rationality
A rational agent chooses the action that gives the **best expected result** based on the information currently available.
The rescue robot should:

- Avoid obstacles
- Avoid unnecessary risk
- Minimize travel cost
- Reach the survivor

- Adapt when new information is discovered

Therefore, selecting a safer low-cost path is a **rational decision**.

3. Environment Type

The rescue environment can be classified as:

Property	**Environment**
Observability	**Partially observable**
Agents	**Single agent**
Deterministic	**Partially stochastic/uncertain**
Episodic/Sequential	**Sequential**
Static/Dynamic	**Dynamic**
State	**Discrete grid**
Known/Unknown	**Initially unknown**

Explanation

Partially observable: The robot sees only nearby cells.
Single agent: The robot independently performs the rescue.
Uncertain: The robot may not initially know all obstacles or risks.
Sequential: Every movement affects future choices.
Dynamic: Disaster conditions can change.
Discrete: The environment is represented using grid cells.
Unknown: The complete map is not available initially.

3(d). Justification of the Approach

The combination of **UCS and Online Search** is appropriate because the robot has **different movement costs and incomplete environmental information**.

Why UCS?

UCS considers cumulative cost:

$$g(n) = \sum \text{movement costs}$$

Therefore, it can distinguish between:

- A longer but safe route
- A shorter but risky route

It chooses the route with the lowest total cost.

Why Online Search?

The robot cannot initially see the complete building.
It must:
Observe → Update → Search → Move → Observe again
This allows it to adapt when it discovers:

- New obstacles
- Risky zones
- Unexpected paths

Final Conclusion

The rescue robot should use an **intelligent online search agent based on Uniform Cost Search**.
Its decision-making can be summarized as:

ENVIRONMENT
↓
Perception
↓
Update Knowledge
↓
Calculate Costs
↓
Uniform Cost Search
↓
Select Best Path
↓
Move
↓
New Observation
↺

Final Answer

Online Intelligent Agent + Uniform Cost Search

is the most suitable approach because it handles **partial observability, obstacles, varying movement costs and dynamically updated knowledge** while aiming to reach the survivor with minimum total cost.